PART 10 — THREAT MODEL & SECURITY PLAN

Security Architecture: Defense-in-Depth Multi-Tenant Isolation

Threat Modeling Standard: Microsoft STRIDE Model & OWASP Top 10 (2025/2026)

Authentication Standard: Node.js native `crypto.scrypt` with constant-time equality

1. Security Philosophy (Non-Technical Explanation)

Accounting software holds a company's most sensitive secrets: profit margins, customer pricing, employee payroll, bank balances, and tax liabilities. If this data is leaked or tampered with, companies can face corporate espionage, extortion, or regulatory penalties under UK GDPR.

blynt approaches security like a high-security bank building:

1. **The Perimeter (HTTPS & Strict CORS):** All data moving across the web is encrypted using TLS 1.3. Only verified company domains can communicate with the server.
2. **The Front Desk (Cryptographic Passwords):** When you log in, your password is not saved in the database. Instead, it is scrambled using a mathematical one-way grinder called `scrypt`. Even if a hacker managed to steal the database, they could not read the passwords.
3. **The Keycard (256-bit Session Tokens):** Once verified, you receive a temporary cryptographically signed keycard that tracks your IP address and device. You can log out of remote devices with a single click.
4. **The Physical Vault Walls (Composite Foreign Keys):** Even if an unauthorized person snuck inside the building, the vault doors between companies are physically separated. Company A's keys cannot turn Company B's lock.

2. STRIDE Threat Analysis & Mitigations

Threat Category	Potential Attack Vector	blynt Architectural Defense
Spoofing (Identity Theft)	Adversary intercepts or guesses session tokens to impersonate an accountant.	Cryptographically strong 256-bit session tokens stored in secure `user_sessions`; IP and user-agent matching; instant token revocation endpoint.
Tampering (Data Manipulation)	Malicious insider attempts to alter an old invoice or journal entry in a filed period.	Three-tier period locking (`HARD_LOCKED`) blocks changes. PostgreSQL append-only triggers prevent `UPDATE` or `DELETE` on `audit_logs`.
Repudiation (Denial of Action)	A bookkeeper makes a fraudulent entry and claims "someone else did it."	Every write, validation, and reversal operation logs the user ID, timestamp, IP address, and unique request ID (`x-request-id`) in the immutable audit log.
Information Disclosure	Multi-tenant SQL parameter injection attempts to leak another company's	Dual-layer isolation: Application-layer tenant scoping AND database-level

	records.	composite foreign keys `(organization_id, id)`.
Denial of Service (DoS)	Attacker floods the API with 100,000 journal lines to exhaust server memory.	NestJS `ValidationPipe` payload limits; NestJS Throttler rate limiting (100 req/min/IP); database query pagination caps.
Elevation of Privilege	A bookkeeper modifies HTTP parameters to grant themselves `OWNER` status.	NestJS `@Roles()` decorator paired with `RolesGuard` validating permissions inside protected transaction blocks.

3. Password Hashing & Cryptographic Mechanics

3.1 Node.js Native `crypto.scrypt`

Rather than relying on outdated MD5, SHA-256, or third-party wrappers, blynt uses Node.js native `crypto.scrypt`:

- **Salt Generation:** 16 bytes of cryptographically secure pseudorandom data generated via `crypto.randomBytes(16)`.
- **Memory & CPU Cost Parameters:** Configured with high memory and iteration cost (`cost: 16384, blockSize: 8, parallel: 1`), providing resistance against GPU/ASIC brute-force dictionary attacks.

3.2 Timing-Safe Equality Verification

Standard string comparison (`password === hash`) terminates early on the first mismatched character. Attackers can measure response times down to nanoseconds to deduce valid password characters (timing side-channel attacks). blynt enforces constant-time comparisons:

`typescript

const isMatch = crypto.timingSafeEqual(storedHashBuffer, computedHashBuffer);

4. Multi-Tenant Foreign Key Hardening (SQL Defense)

Standard multi-tenant systems use single-column foreign keys (`contact_id` pointing to `contacts.id`). If application code makes a mistake, a record in Company A can point to a contact in Company B.

blynt resolves this by enforcing composite foreign keys:

`sql

ALTER TABLE "journal_lines"

ADD CONSTRAINT "journal_lines_organization_id_contact_id_fkey"

FOREIGN KEY ("organization_id", "contact_id")

REFERENCES "contacts"("organization_id", "id")

ON DELETE RESTRICT ON UPDATE CASCADE;

If an adversary crafts a payload referencing `contact_id` belonging to Tenant B while authenticated in Tenant A, PostgreSQL terminates the transaction with foreign key violation code `23503`.